Lab No. 4

Simple Python Database Connections

NAME: LAIBA RANI

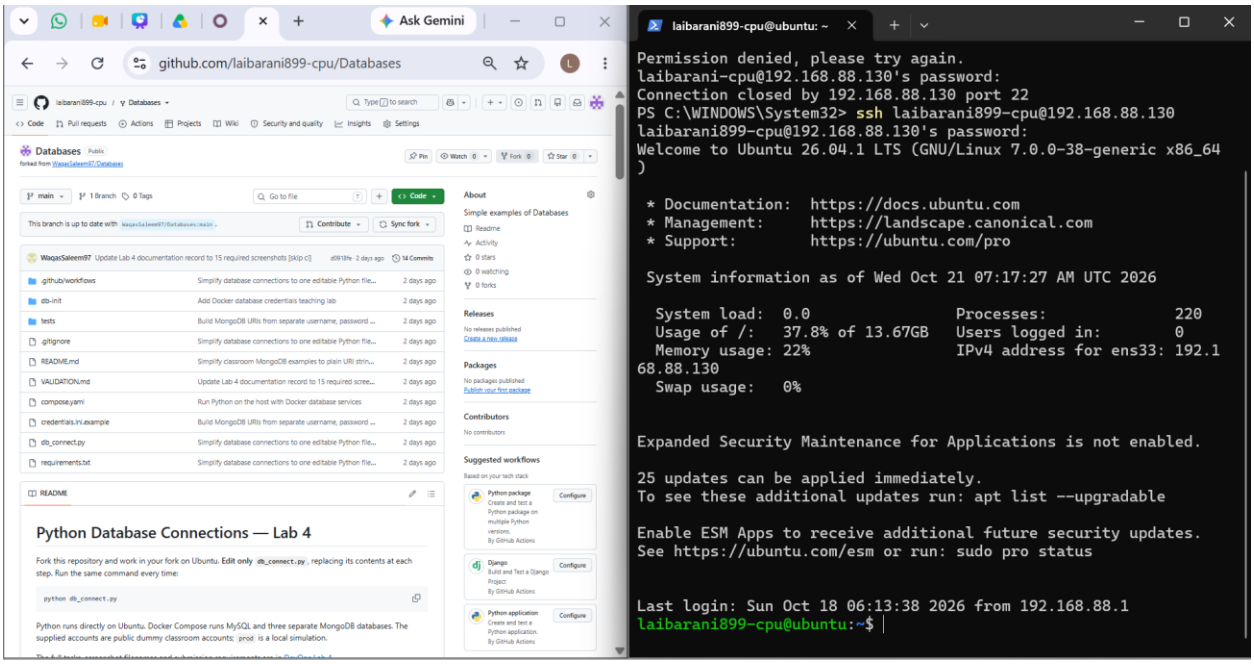
REGISTRATION NO.: 2023-BCS-053

COURSE CODE: CS-649

SECTION: A GITHUB

USERNAME: laibarani899-cpu

Task 1: Fork and Clone the Databases Repository



Open Databases, select **Fork**, and keep the name `Databases`

```
laibarani899-cpu@ubuntu:~$ cd ~
git clone git@github.com:laibarani899-cpu/Databases.git
Cloning into 'Databases'...
remote: Enumerating objects: 101, done.
remote: Counting objects: 100% (101/101), done.
remote: Compressing objects: 100% (90/90), done.
remote: Total 101 (delta 30), reused 1 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (101/101), 39.86 KiB | 396.00 KiB/s, done.
Resolving deltas: 100% (30/30), done.
laibarani899-cpu@ubuntu:~$ cd ~/Databases
git remote -v
origin  git@github.com:laibarani899-cpu/Databases.git (fetch)
origin  git@github.com:laibarani899-cpu/Databases.git (push)
```

Clone your own fork:

Task 2: Install Docker, Docker Compose, and Python on Ubuntu

```
laibarani899-cpu@ubuntu: ~
C:\Users\PC>ssh laibarani899-cpu@192.168.88.130
laibarani899-cpu@192.168.88.130's password:
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-38-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Thu Oct 22 09:48:49 AM UTC 2026

  System load:  1.38               Processes:             282
  Usage of /:   75.5% of 13.67GB   Users logged in:       0
  Memory usage: 39%                IPv4 address for ens33: 192.168.88.130
  Swap usage:   0%

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

   https://ubuntu.com/workshop

Expanded Security Maintenance for Applications is not enabled.

25 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm


Last login: Thu Oct 22 08:21:32 2026 from 192.168.88.1
laibarani899-cpu@ubuntu:~$ git --version
git version 2.53.0
laibarani899-cpu@ubuntu:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
laibarani899-cpu@ubuntu:~$ docker compose version
Docker Compose version v5.6.0
```

Reconnect and verify the Git, Docker, and Docker Compose installations

Task 3: Start the Database Services

```
laibarani899-cpu@ubuntu:~/Databases$ docker compose up -d --wait mysql mongo-dev mongo-test mongo-prod
[+] up 4/4
 ✔Container databases-mongo-dev-1  Healthy                                                    0.6s
 ✔Container databases-mongo-test-1 Healthy                                                    0.6s
 ✔Container databases-mongo-prod-1 Healthy                                                    0.6s
 ✔Container databases-mysql-1      Healthy                                                    0.6s
laibarani899-cpu@ubuntu:~/Databases$ docker compose ps
NAME                IMAGE       COMMAND                  SERVICE      CREATED        STATUS                    PORTS
databases-mongo-dev-1    mongo:8.2   "docker-entrypoint.s…"   mongo-dev    25 hours ago   Up 14 minutes (healthy)   127.0.0.1:27017->
27017/tcp
databases-mongo-prod-1   mongo:8.2   "docker-entrypoint.s…"   mongo-prod   25 hours ago   Up 5 minutes (healthy)    127.0.0.1:27019->
27017/tcp
databases-mongo-test-1   mongo:8.2   "docker-entrypoint.s…"   mongo-test   25 hours ago   Up 7 minutes (healthy)    127.0.0.1:27018->
27017/tcp
databases-mysql-1        mysql:8.4   "docker-entrypoint.s…"   mysql        25 hours ago   Up 28 minutes (healthy)   33060/tcp, 127.0.
0.1:3307->3306/tcp
laibarani899-cpu@ubuntu:~/Databases$
```

Start the four supplied databases:

```
laibarani899-cpu@ubuntu:~/Databases$ docker compose port mysql 3306
127.0.0.1:3307
laibarani899-cpu@ubuntu:~/Databases$ docker compose port mongo-dev 27017
127.0.0.1:27017
laibarani899-cpu@ubuntu:~/Databases$ docker compose port mongo-test 27017
127.0.0.1:27018
laibarani899-cpu@ubuntu:~/Databases$ docker compose port mongo-prod 27017
127.0.0.1:27019
```

Inspect the ports used by Python on the Ubuntu host

Task 4: MySQL with Values in the Code

```
(.venv) laibarani899-cpu@ubuntu:~/Databases$ python db_connect.py
Connected to MySQL: school_mysql
Server: 127.0.0.1 3307
Environment: dev
(.venv) laibarani899-cpu@ubuntu:~/Databases$
```

Run the file

the MySQL connection was successful using hardcoded connection settings in Python. The program connected to `school_mysql` through `127.0.0.1:3307` and retrieved the environment value `dev`.

Task 5: MongoDB with Values in the Code

```
(.venv) laibarani899-cpu@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) laibarani899-cpu@ubuntu:~/Databases$
```

Replace the contents of **the same** **`db_connect.py`**

`database.command("ping")` sends a real request; creating a client or selecting a database alone does not verify connectivity. The final query reads the environment from the selected database. `serverSelectionTimeoutMS=5000` limits server selection to five seconds.

Task 6: MongoDB with os.getenv()

```
(.venv) laibarani899-cpu@ubuntu:~/Databases$ export MONGO_USERNAME='dev_user'
export MONGO_PASSWORD='DevDemoOnly123!'
export MONGO_ENDPOINT='127.0.0.1:27017'
export MONGO_DATABASE='school_dev'

python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) laibarani899-cpu@ubuntu:~/Databases$
```

Set the dev values in the same Ubuntu shell

```
(.venv) laibarani899-cpu@ubuntu:~/Databases$ export MONGO_USERNAME='test_user'
export MONGO_PASSWORD='TestDemoOnly123!'
export MONGO_ENDPOINT='127.0.0.1:27018'
export MONGO_DATABASE='school_test'
python db_connect.py
Connected to MongoDB: school_test
Server: ('127.0.0.1', 27018)
Environment: test
(.venv) laibarani899-cpu@ubuntu:~/Databases$
```

Set the test values and run

```
(.venv) laibarani899-cpu@ubuntu:~/Databases$ export MONGO_USERNAME='prod_user'
export MONGO_PASSWORD='ProdDemoOnly123!'
export MONGO_ENDPOINT='127.0.0.1:27019'
export MONGO_DATABASE='school_prod'

python db_connect.py
Connected to MongoDB: school_prod
Server: ('127.0.0.1', 27019)
Environment: prod
(.venv) laibarani899-cpu@ubuntu:~/Databases$
```

Set the simulated prod values and run

Task 7: MongoDB with a Credentials File

```
(.venv) laibarani899-cpu@ubuntu:~/Databases$ cp -n credentials.ini.example credentials.ini
ls credentials.ini
(.venv) laibarani899-cpu@ubuntu:~/Databases$ chmod 600 credentials.ini
stat -c '%a %n' credentials.ini
600 credentials.ini
(.venv) laibarani899-cpu@ubuntu:~/Databases$
```

Create one local file containing the three environment section

Restrict the file to your account and check its mode

```
(.venv) laibarani899-cpu@ubuntu:~/Databases$ export APP_ENV=dev
python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
```

Choose dev and run

```
(.venv) laibarani899-cpu@ubuntu:~/Databases$ export APP_ENV=test
python db_connect.py
Connected to MongoDB: school_test
Server: ('127.0.0.1', 27018)
Environment: test
```

Choose test and run

```
(.venv) laibarani899-cpu@ubuntu:~/Databases$ export APP_ENV=prod
python db_connect.py
Connected to MongoDB: school_prod
Server: ('127.0.0.1', 27019)
Environment: prod
```

Choose simulated prod and run

Link Of My Fork Repository:

https://github.com/laibarani899-cpu/Databases